

SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing
and Big Data
Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan k

Assessment Tasks Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.
Design a cloud architecture using scalability and caching techniques.
Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.
2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments.
Propose a solution using container orchestration and multi-cloud strategies.
Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.
3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy.
Design a solution using cloud-native backup, replication, and versioning.
Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.
4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.
Design a hybrid architecture that ensures secure communication between environments.
Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.
5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
The system must continue operating even if one region goes down.
Propose a multi-region deployment strategy in the cloud.
Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and wellorganized response	Generally clear	Somewhat unclear	Poorly presented

Question 1) Video Streaming Platform Migration to the Cloud Scenario

A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques, explain how load balancing and auto-scaling ensure smooth streaming, and discuss cost optimization strategies for high-demand workloads.

1. Cloud Architecture Using Scalability and Caching Techniques

The platform should adopt a microservices-based, horizontally scalable architecture deployed across multiple availability zones. Video content is ingested, transcoded into adaptive bitrate formats (e.g., HLS/DASH) using elastic transcoding services, and stored in object storage (e.g., Amazon S3 or Azure Blob Storage) for durability and virtually unlimited capacity.

- **Edge caching / CDN:** A Content Delivery Network (CDN) with edge nodes caches popular video segments close to end users, drastically reducing origin-server load and start-up latency.
- **Application-layer caching:** An in-memory distributed cache (e.g., Redis or Memcached) stores frequently accessed metadata, user session data, and recommendation results to reduce database round-trips.
- **Stateless microservices:** Stateless microservices (authentication, catalog, recommendation, playback) run in containers so any instance can serve any request, enabling seamless horizontal scaling.
- **Event-driven decoupling:** A message queue (e.g., Kafka/SQS) decouples ingestion, transcoding, and analytics pipelines so spikes in one stage do not cascade into others.

2. Role of Load Balancing and Auto-Scaling

A global/DNS-based load balancer (e.g., AWS Global Accelerator or a GeoDNS service) routes users to the nearest healthy regional cluster, minimizing latency and buffering. Within each region, an application load balancer distributes incoming requests across a fleet of stateless service instances using health checks, so no single instance becomes a bottleneck.

Auto-scaling groups monitor real-time metrics (CPU utilization, request rate, network throughput) and automatically add instances during traffic surges (e.g., a live sports event) and remove them during off-peak hours. Combined with the CDN offloading the majority of read traffic, this ensures playback remains smooth even under millions of concurrent viewers, while origin servers are protected from overload.

3. Cost Optimization Strategies

- **Reduce origin load:** Use CDN edge caching aggressively to minimize expensive origin egress traffic and repeated transcoding.
- **Mixed pricing models:** Combine Reserved/Savings-Plan instances for predictable baseline load with Spot/Preemptible instances for batch transcoding jobs that can tolerate interruption.
- **Storage tiering:** Apply tiered storage — hot (frequently viewed) content on high-performance storage, cold (rarely viewed) content on lower-cost archival storage.
- **Scale-down and serverless:** Use auto-scaling with scale-to-zero for non-critical microservices during idle periods, and serverless functions for bursty, short-lived tasks (e.g., thumbnail generation).
- **Cost monitoring:** Continuously monitor cost dashboards and set budget alerts to catch anomalies such as runaway transcoding jobs.

Justification

This design directly targets the two stated requirements — massive concurrency and minimal buffering — by offloading most traffic to the CDN and cache layers (performance), while auto-scaling and mixed pricing models keep infrastructure spend proportional to actual demand (cost efficiency), making the architecture both technically robust and economically sustainable.

Question 2) Multi-Cloud Container Orchestration for a Fintech Startup Scenario

A fintech startup deploys applications using containers across multiple cloud providers and faces challenges maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies, explain service discovery and scaling, and highlight benefits and risks of a multi-cloud deployment model.

1. Proposed Solution — Container Orchestration and Multi-Cloud Strategy

Kubernetes should be adopted as the common orchestration layer since it is cloud-agnostic and supported natively (EKS, AKS, GKE) or self-managed on every major provider. A GitOps-based CI/CD pipeline (e.g., ArgoCD/Flux with Terraform for infrastructure-as-code) ensures identical container images, configuration, and deployment manifests are applied consistently across all clusters, eliminating configuration drift between providers.

- **Unified image registry:** Use a container registry mirrored across providers (or a multi-cloud-aware registry) so images are consistently available regardless of which cloud a workload runs on.
- **Configuration abstraction:** Standardize on Helm charts / Kustomize overlays so provider-specific differences (storage classes, networking) are isolated from application logic.
- **Service mesh:** Adopt a service mesh (e.g., Istio or Linkerd) spanning clusters to provide uniform traffic management, mTLS security, and observability across clouds.

2. Service Discovery and Scaling

Within each Kubernetes cluster, built-in DNS-based service discovery lets pods locate each other automatically as they scale up or down. For cross-cloud discovery, the service mesh maintains a global service registry so a request originating in one cloud can be routed to a healthy service instance in another, enabling active-active deployment.

The Horizontal Pod Autoscaler (HPA) scales pods based on CPU/memory or custom metrics (e.g., transaction queue length), while the Cluster Autoscaler provisions additional nodes as needed. Because this behavior is defined declaratively, the same scaling policy applies uniformly whether the workload is running on Provider A or Provider B, ensuring consistent performance for latency-sensitive financial transactions.

3. Benefits and Risks of Multi-Cloud Deployment

- **Benefit — Flexibility:** Avoids vendor lock-in and increases negotiating leverage with providers.
- **Benefit — Resilience:** Improves resilience: an outage at one provider does not take down the entire platform, which is critical for financial-services availability requirements.
- **Benefit — Compliance & cost:** Allows workload placement based on regulatory data-residency requirements or cost differences between providers.

- **Risk — Complexity:** Increased operational complexity — teams must manage differing IAM models, networking constructs, and SLAs across providers.
- **Risk — Cost & latency:** Higher inter-cloud data transfer (egress) costs and potential latency when services communicate across providers.
- **Risk — Security consistency:** A larger, more heterogeneous attack surface requires consistent security policy enforcement, which is harder across differing native security tools.

Justification

Standardizing on Kubernetes and GitOps directly resolves the consistency problem described in the scenario, while the service mesh and autoscalers provide the uniform discovery and scaling behavior a fintech platform needs for reliable, low-latency transaction processing — with the risk trade-offs made explicit so the startup can plan mitigations such as centralized policy management.

Question 3) Backup and Disaster Recovery for an E-Learning Platform Scenario

An e-learning platform experiences data loss due to accidental deletion in the cloud and needs a robust backup and disaster recovery (DR) strategy. Design a solution using cloud-native backup, replication, and versioning; explain how RTO and RPO are achieved; and provide steps to ensure business continuity during failures.

1. Backup, Replication, and Versioning Design

- **Versioning:** Enable object versioning on cloud storage so every prior version of a file is retained; accidental deletion becomes recoverable via a simple restore rather than a permanent loss.
- **Automated backups:** Schedule automated, incremental snapshots of databases and file systems (e.g., daily full + hourly incremental) using native backup services (AWS Backup, Azure Backup).
- **Cross-region replication:** Apply cross-region replication so a copy of all critical data continuously syncs to a geographically separate region.
- **Retention & delete protection:** Enforce a retention/lifecycle policy (e.g., 30 daily, 12 monthly snapshots) and enable MFA-delete / soft-delete protections so backups themselves cannot be casually removed.
- **Restore testing:** Regularly test restores in an isolated environment to confirm backups are actually usable, not just present.

2. Achieving RTO and RPO Targets

Recovery Point Objective (RPO) — the maximum tolerable data loss — is achieved by shortening the interval between backups/replication events; near-continuous replication (e.g., database transaction-log shipping every few minutes) can bring RPO down to minutes rather than a full day. Recovery Time Objective (RTO) — the maximum tolerable downtime — is achieved by maintaining a warm standby environment in the secondary region (pre-provisioned infrastructure via infrastructure-as-code) so failover is a matter of redirecting DNS/traffic rather than rebuilding systems from scratch, and by automating the failover process to remove manual delay.

3. Steps to Ensure Business Continuity During Failures

- **Detect:** Detect the failure quickly using automated health checks and monitoring/alerting.
- **Failover:** Trigger an automated or semi-automated failover to the replicated secondary region using pre-tested runbooks.
- **Reroute traffic:** Redirect user traffic via DNS/load-balancer updates to the standby environment.
- **Restore:** Restore the affected data from the most recent clean backup/version once the root cause is identified.

- **Review & harden:** Conduct a post-incident review and update the DR plan and access controls (e.g., restrict delete permissions, require approval workflows) to prevent recurrence.

Justification

Versioning directly solves the accidental-deletion problem described, replication and standby infrastructure minimize RTO, and frequent incremental backups minimize RPO — together giving the platform a measurable, testable continuity guarantee rather than an ad-hoc recovery process.

Question 4) Hybrid Cloud Model for a Government Agency Scenario

A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data; some applications must remain on-premises for compliance reasons. Design a hybrid architecture ensuring secure communication between environments, explain identity management and encryption implementation, and discuss challenges in managing hybrid cloud environments.

1. Hybrid Architecture Design

Sensitive, regulated workloads (citizen records, classified data) remain on-premises or in a private cloud, while non-sensitive workloads (public information portals, non-critical analytics) run on a public cloud to benefit from elasticity and cost efficiency. The two environments are connected via a dedicated, private network link (e.g., AWS Direct Connect / Azure ExpressRoute or an IPsec VPN as backup), avoiding exposure over the public internet.

- **Secure connectivity:** A VPN/dedicated-line connection with redundant paths links the on-premises data center to the cloud VPC/VNet, isolated in its own private subnet with no direct public exposure.
- **Network segmentation:** A Web Application Firewall (WAF) and network segmentation (security groups, NACLs) restrict traffic between tiers to only what is explicitly required.
- **Controlled integration:** A hybrid API gateway mediates and logs every request that crosses the on-premises/cloud boundary.

2. Identity Management and Encryption

A federated identity solution (e.g., Active Directory integrated with a cloud identity provider via SAML/OIDC) provides a single source of truth for user identity across both environments, enforcing multi-factor authentication and role-based access control (RBAC) so users and services only access what their role permits (principle of least privilege).

Encryption is applied at every layer: data at rest is encrypted using AES-256 with keys managed in a dedicated key-management service (cloud KMS or on-premises HSM, with

the agency retaining control of key material for sensitive data); data in transit between on-premises and cloud is protected using TLS 1.2+/IPsec over the dedicated link.

3. Challenges in Managing Hybrid Cloud Environments

- **Policy consistency:** Maintaining consistent security policy and compliance posture (e.g., FedRAMP-style controls) across two fundamentally different environments.
- **Latency:** Network latency and bandwidth constraints on the link between on-premises and cloud can affect application performance.
- **Observability:** Unified monitoring and logging is harder when telemetry sources are split across environments, complicating incident detection.
- **Operational skills:** Skill gaps — staff need expertise in both traditional data-center operations and cloud-native tooling.
- **Data governance:** Data classification errors (placing sensitive data in the wrong environment) pose a serious compliance risk and require strict governance.

Justification

Keeping regulated data on-premises while offloading non-sensitive workloads to the cloud satisfies the compliance constraint stated in the scenario; federated identity and layered encryption secure the boundary between the two environments, and the challenges are called out explicitly so the agency can plan governance and monitoring investments up front.

Question 5)Multi-Region High Availability for a SaaS Provide Scenario

A SaaS provider needs high availability and fault tolerance so the application continues operating even if one region goes down. Propose a multi-region deployment strategy, explain data replication and failover mechanisms, and discuss monitoring and alerting strategies for proactive issue detection.

1. Multi-Region Deployment Strategy

The application is deployed in an active-active configuration across at least two geographically separated regions, each running a full, independent stack (compute, database replica, cache) behind its own load balancer. A global traffic manager (e.g., Route 53 with health checks, or a global load balancer) routes users to the nearest healthy region under normal conditions and automatically withdraws traffic from an unhealthy region.

- **Infrastructure as code:** Infrastructure is defined as code (Terraform/CloudFormation) so both regions are provisioned identically and can be rebuilt or scaled consistently.
- **Stateless compute per region:** Stateless application tiers in each region can serve traffic independently; only the data tier requires careful cross-region coordination.

2. Data Replication and Failover Mechanisms

The primary database uses synchronous replication within a region (for zero data loss on a single-node failure) and asynchronous cross-region replication to the secondary region (to avoid the latency penalty of synchronous writes over long distances). Managed multi-region database services (e.g., Aurora Global Database, Cosmos DB multi-region writes) automate this replication and expose a defined replication lag that determines the achievable RPO.

Failover is automated: health checks continuously probe each region, and if the primary region becomes unreachable, the traffic manager reroutes users to the secondary region while the database service promotes the secondary replica to primary. DNS TTLs are kept low to speed propagation, and the failover process is regularly tested (chaos engineering / game days) to validate that RTO targets are actually met in practice.

3. Monitoring and Alerting for Proactive Detection

- **Centralized observability:** Centralized observability stack (metrics, logs, distributed traces) aggregates telemetry from all regions into a single dashboard.
- **Synthetic monitoring:** Synthetic transaction monitoring continuously simulates real user journeys from multiple geographies to catch degradation before real users report it.
- **Alerting:** Threshold- and anomaly-based alerts (latency, error rate, replication lag, resource saturation) trigger automated notifications to on-call teams via a paging system.
- **Automated remediation:** Automated runbooks/self-healing actions (e.g., auto-restart, auto-scale) resolve common issues without waiting for human intervention.
- **Continuous validation:** Regular game-day drills validate that alerts, dashboards, and failover automation work together as expected under simulated regional failure.

Justification

The active-active multi-region design directly satisfies the requirement that the system keep operating if one region fails; combining synchronous intra-region replication with asynchronous cross-region replication balances durability against latency, and proactive synthetic monitoring plus automated failover minimizes both detection time and recovery time, giving measurable, defensible RTO/RPO outcomes.